UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE CIÊNCIAS DA COMPUTAÇÃO

T168: Álgebra e Geometria Computacional

INTRODUÇÃO

Aqui estão algumas notas gerais e ideias que irão ajudá-lo com o **trabalho de implementação**. O objetivo do **trabalho de implementação** é expor você a problemas que são mais complicados e abertos do que os tradicionais exercícios de Álgebra Linear. Eles vão exigir que você realmente entenda algumas aplicações da Álgebra Linear, muito mais do que você aprenderia resolvendo exercícios teóricos de Álgebra Linear ems ala de aula. Estamos fazendo isso porque aprender a resolver esse tipo de problema é uma excelente preparação para os desafios que você terá pela frente e para a carreira que você seguirá, uma vez que sair da universidade. Abaixo seguem algumas observações gerais sobre o trabalho. Isso fará ainda mais sentido depois de ter feito os exercícios recomendados, pois eles são bons indicadores para se preparar para o trabalho pela frente.

- **Não desista facilmente (e não tenha medo da complexidade)!**
  Todo problema que alguém lhe pagar para resolver (independente do seu cargo ou carreira) será complicado, confuso e difícil de definir. Todos os problemas fáceis são agora realizados por computadores e robôs. Então agora é a hora de começar a aprender a lidar com problemas mais complexos. Isto é muito diferente de aprender a resolver questões de provas de "Conhecimento matemático" ou "capacidade matemática".

- **Leia as instruções (mais de uma vez!)**
  Você pode ter que ler as instruções várias vezes antes de entender a pergunta. Isto é algo esperado! Muitas vezes ler as instruções em voz alta ajuda no processo de entendimento.

- **Converse com seus colegas (mesmo que nenhum de vocês saiba a resposta)!**
  É fácil pensar sobre uma conversa como um processo onde o conhecimento é transmitido de um interlocutor para outro. Isto geralmente é verdade. Contudo conversar pode ser uma maneira de um grupo de pessoas criar um novo conhecimento, de forma cooperativa, que nenhum tinha antes do início da conversa. Nessas conversas, geralmente não existe a pessoa com os conhecimentos avançados necessários para realizar uma contribuição importante. Porém, nesse momento, realizar as perguntas certas pode ser muito mais importante e produtivo do que possuir uma pessoa capaz dar uma solução. Aprender como ter esse tipo de conversa é uma das coisas mais importantes que você pode aprender neste tipo de atividade.

ATIVIDADE AVALIATIVA - AV1

O objetivo deste trabalho é implementar, utilizando uma das seguintes linguagens de programação JavaScript, Java, C, C++, C#, Python e etc., a representação de Matriz, Vetor, as operações algébricas elementares, eliminação gaussiana e o algoritmo Gauss-Jordan. Mais explicitamente, deve-se implementar:

- Classe Matrix

  Implementar uma classe nomeada *Matrix* que implemente os seguintes itens:

  - Construtor da classe *Matrix* que receba como parâmetros a quantidade de linhas, colunas e um array com os elementos da matriz. A assinatura do construtor deve seguir a seguinte estrutura:

```
constructor(rows, cols, elements){}

rows        -> A quantidade de linhas da matriz.
cols        -> A quantidade de colunas da matriz.
elements    -> Um array com os elementos da matriz.
```

  - Um método chamado **get** que tem como função retornar o valor armazenado em uma determinada posição da matriz. O método recebe como parâmetros a linha e a coluna da posição do elemento da matriz e retorna o valor armazenado nesta posição. A assinatura da função **get** deve seguir a seguinte estrutura:

```
get(i, j){}

i -> O índice da linha da matriz.
j -> O índice da coluna da matriz.

Retorno -> O valor do elemento que está na linha i e coluna j
da matriz.
```

  - Um método chamado **set** que tem como função atribuir um valor em uma determinada posição da matriz. O método recebe como parâmetros a linha, a coluna e o valor do elemento a ser armazenado na matriz. A assinatura da função **set** deve seguir a seguinte estrutura:

```
set(i, j, value){}

i     -> O índice da linha da matriz.
j     -> O índice da coluna da matriz.
value -> O valor a ser armazenado na matriz.

Retorno -> Nenhum.
```

- Classe Vector

  Implementar uma classe nomeada *Vector* que implemente os seguintes itens:

  - Construtor da classe *Vector* que receba como parâmetros a dimensão do vetor e um array com os elementos do vetor. A assinatura do construtor deve seguir a seguinte estrutura:

```
constructor(dim, elements){}

dim         -> Um número que representa a dimensão do Vetor
elements    -> Um array que contém os elementos do vetor
```

  - Um método chamado **get** que tem como função retornar o valor armazenado em uma determinada posição do vetor. O método recebe como parâmetros o índice do elemento do

vetor e retorna o valor armazenado nesta posição. A assinatura da função **get** deve seguir a seguinte estrutura:

```
get(i){}

i     -> O índice do elemento que deseja-se obter.

Retorno -> O valor do elemento que está no índice i do vetor.
```

- Um método chamado **set** que tem como função atribuir um valor em uma determinada posição do vetor. O método recebe como parâmetros o índice do elemento do vetor e o valor a ser armazenado no vetor. A assinatura da função **set** deve seguir a seguinte estrutura:

```
set(i, value){}

i     -> O índice do elemento que deseja-se obter.
value -> O valor a ser armazenado no vetor.

Retorno -> Nenhum.
```

- Classe *LinearAlgebra*

  Implementar uma classe nomeada *LinearAlgebra* que implemente os seguintes itens:

  - Um método chamado **transpose** que tem como função retornar uma matriz transposta ou um vetor transposto. O método deve receber como parâmetro um objeto da classe *Matrix* ou um objeto da classe *Vector*. A assinatura da função **transpose** deve seguir a seguinte estrutura:

```
transpose(a){}

a -> A matriz ou o vetor a ser transposto.

Retorno -> Uma matriz ou o vetor transposto.
```

  - Um método chamado **sum** que tem como função somar duas matrizes ou dois vetores. O método deve receber como parâmetro dois objetos da classe *Matrix* ou dois objetos da classe *Vector*. A assinatura da função **sum** deve seguir a seguinte estrutura:

```
sum(a, b){}

a, b -> A matriz ou o vetor a ser somado.

Retorno -> Uma matriz ou vetor resultante da soma.
```

  - Um método chamado **times** que tem como função multiplicar duas matrizes ou dois vetores ou um escalar com uma Matrix. O método deve receber como parâmetro dois objetos da classe *Matrix* ou dois objetos da classe *Vector*. A assinatura da função **times** deve seguir a seguinte estrutura:

```
times(a, b){}

a -> Pode ser um escalar (número) ou um objeto do tipo Matrix
ou um objeto do tipo Vector.
b -> Um objeto do tipo Matrix ou um objeto do tipo Vector.
```

```
Retorno -> Uma matriz ou vetor resultante da multiplicação.
```

- Um método chamado **gauss** que tem como função realizar a eliminação gaussiana em uma Matriz. O método deve receber como parâmetro um objeto da classe *Matrix* e retornar a *Matrix* resultante da operação de eliminação gaussiana. A assinatura da função **gauss** deve seguir a seguinte estrutura:

```
gauss(a){}

a -> A matriz a qual deve-se realizar a eliminação gaussiana.

Retorno -> Uma matriz com o resultado da eliminação gaussiana.
```

- Um método chamado **solve** que tem como função um sistema de equações lineares. O método deve receber como parâmetro um objeto da classe *Matrix* (que representa uma matriz aumentada) e retornar a *Matrix* resultante da resolução do sistema linear. A assinatura da função **solve** deve seguir a seguinte estrutura:

```
solve(a){}

a -> A matriz aumentada na qual deve-se realizar a eliminação
gaussiana.

Retorno -> Uma matriz com o resultado do sistema de equações
lineares.
```

Deve-se implementar os tratamentos de erros das funções. Fique a vontade para adicionar métodos que não foram listados acima, com o intuito de organizar o código e de colocar situações de verificações de erro onde achar conveniente. Suas idéias e propostas de melhorias serão bem vindas!

Um arquivo **ZIP** contendo o código fonte da **implementação** deve ser entregue através do AVA até a data estipulada. **Não serão aceitos outros métodos de submissão do trabalho. Envios fora do prazo estipulado serão aceitos mas terão desconto de um ponto na nota por dia de atraso**. Quaisquer dúvidas ou pontos não abordados nesse documento devem ser dirimidos com o professor da disciplina antes do término do prazo de submissão do trabalho via AVA(Moodle). Caso contrário, prevalecerá a interpretação deste documento na ótica do professor da disciplina.

"Não tente.
Faça ou não faça.
Não existe Tentativa"